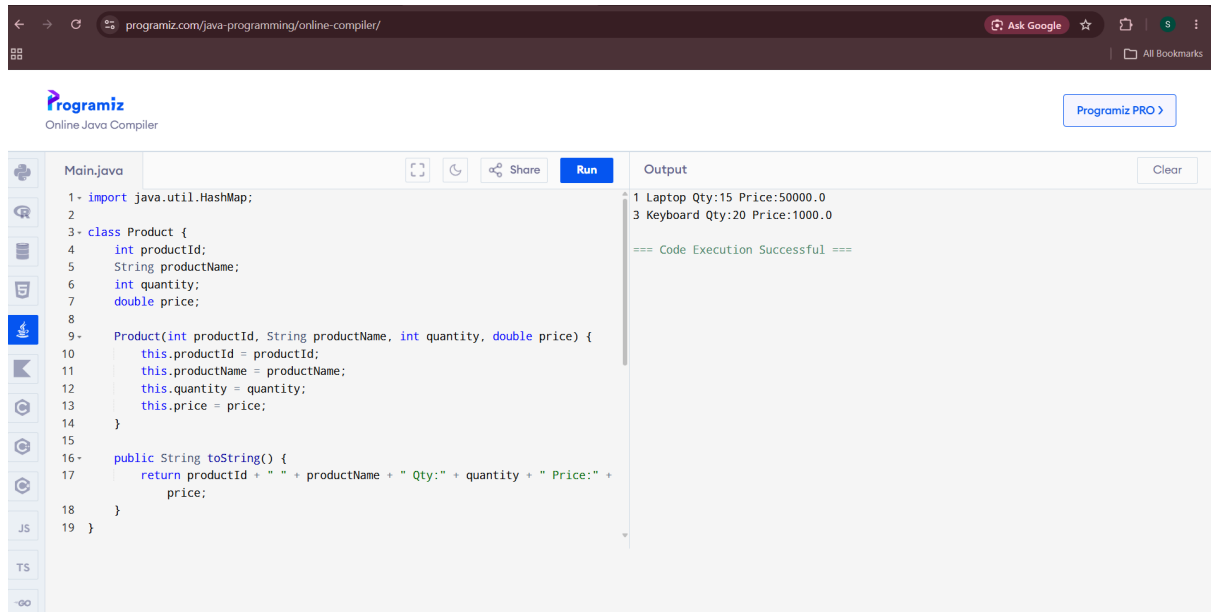


DSA

1. Inventory Management System

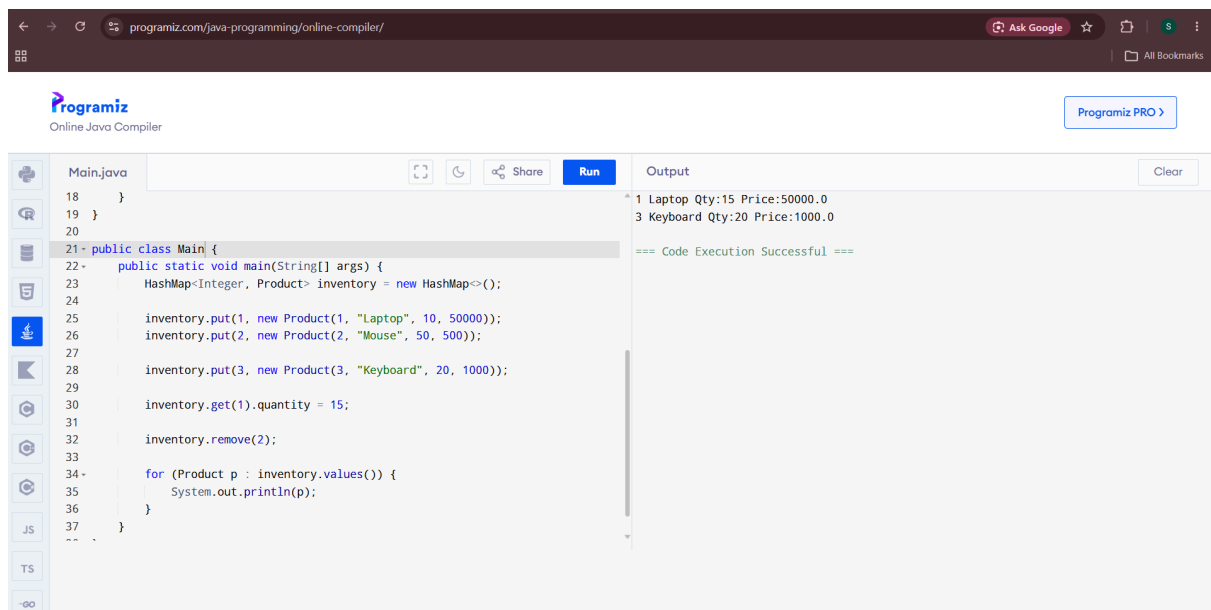


The screenshot shows the Programiz Online Java Compiler interface. The browser address bar displays `programiz.com/java-programming/online-compiler/`. The compiler logo and "Online Java Compiler" text are at the top left. A "Programiz PRO" button is at the top right. The editor area, titled "Main.java", contains the following code:

```
1- import java.util.HashMap;
2
3- class Product {
4     int productId;
5     String productName;
6     int quantity;
7     double price;
8
9-     Product(int productId, String productName, int quantity, double price) {
10        this.productId = productId;
11        this.productName = productName;
12        this.quantity = quantity;
13        this.price = price;
14    }
15
16-     public String toString() {
17        return productId + " " + productName + " Qty:" + quantity + " Price:" +
18            price;
19    }
20 }
```

The "Run" button is highlighted. The output area on the right shows the following text:

```
1 Laptop Qty:15 Price:50000.0
3 Keyboard Qty:20 Price:1000.0
=== Code Execution Successful ===
```



The screenshot shows the same Programiz Online Java Compiler interface. The editor area now contains the following code:

```
18 }
19 }
20
21- public class Main {
22-     public static void main(String[] args) {
23         HashMap<Integer, Product> inventory = new HashMap<>();
24
25         inventory.put(1, new Product(1, "Laptop", 10, 50000));
26         inventory.put(2, new Product(2, "Mouse", 50, 500));
27
28         inventory.put(3, new Product(3, "Keyboard", 20, 1000));
29
30         inventory.get(1).quantity = 15;
31
32         inventory.remove(2);
33
34         for (Product p : inventory.values()) {
35             System.out.println(p);
36         }
37     }
38 }
```

The "Run" button is highlighted. The output area on the right shows the following text:

```
1 Laptop Qty:15 Price:50000.0
3 Keyboard Qty:20 Price:1000.0
=== Code Execution Successful ===
```

2. E-commerce Platform Search Function

Programiz Online Java Compiler

Programiz PRO >

Main.java

```
1- class Product {
2-     int productId;
3-     String productName;
4-     String category;
5-
6-     Product(int id, String name, String category) {
7-         this.productId = id;
8-         this.productName = name;
9-         this.category = category;
10-    }
11- }
12-
13- public class Main {
14-
15-     static int linearSearch(Product[] products, String name) {
16-         for (int i = 0; i < products.length; i++) {
17-             if (products[i].productName.equalsIgnoreCase(name))
18-                 return i;
19-         }
20-         return -1;
21-     }
22- }
```

Output

```
Linear Search: 1
Binary Search: 1
=== Code Execution Successful ===
```

programiz.com/java-programming/online-compiler/

Programiz Online Java Compiler

Programiz PRO >

Main.java

```
20-         return -1;
21-     }
22-
23-     static int binarySearch(Product[] products, String name) {
24-         int low = 0, high = products.length - 1;
25-
26-         while (low <= high) {
27-             int mid = (low + high) / 2;
28-
29-             int result = products[mid].productName.compareToIgnoreCase(name);
30-
31-             if (result == 0)
32-                 return mid;
33-             else if (result < 0)
34-                 low = mid + 1;
35-             else
36-                 high = mid - 1;
37-         }
38-         return -1;
39-     }
40- }
```

Output

```
Linear Search: 1
Binary Search: 1
=== Code Execution Successful ===
```

The screenshot shows the Programiz Online Java Compiler interface. The code editor contains a Java program for binary search. The code is as follows:

```
35         else
36             high = mid - 1;
37     }
38     return -1;
39 }
40
41 public static void main(String[] args) {
42     Product[] products = {
43         new Product(1, "Keyboard", "Electronics"),
44         new Product(2, "Laptop", "Electronics"),
45         new Product(3, "Mouse", "Electronics")
46     };
47
48     System.out.println("Linear Search: " +
49         linearSearch(products, "Laptop"));
50
51     System.out.println("Binary Search: " +
52         binarySearch(products, "Laptop"));
53 }
54 }
```

The output panel shows the results of the execution:

```
Linear Search: 1
Binary Search: 1
=== Code Execution Successful ===
```

3. Sorting Customer Orders

The screenshot shows the Programiz Online Java Compiler interface. The code editor contains a Java program for sorting customer orders using bubble sort. The code is as follows:

```
1- class Order {
2-     int orderId;
3-     String customerName;
4-     double totalPrice;
5-
6-     Order(int id, String name, double price) {
7-         orderId = id;
8-         customerName = name;
9-         totalPrice = price;
10-    }
11- }
12-
13- public class Main {
14-
15-     static void bubbleSort(Order[] orders) {
16-         int n = orders.length;
17-
18-         for (int i = 0; i < n - 1; i++) {
19-             for (int j = 0; j < n - i - 1; j++) {
20-                 if (orders[j].totalPrice > orders[j + 1].totalPrice) {
```

The output panel shows the results of the execution:

```
Ram 1500.0
John 3000.0
Sam 5000.0
=== Code Execution Successful ===
```

programiz.com/java-programming/online-compiler/

Programiz
Online Java Compiler

Programiz PRO >

Main.java

```
21      Order temp = orders[j];
22      orders[j] = orders[j + 1];
23      orders[j + 1] = temp;
24  }
25  }
26  }
27  }
28
29- static int partition(Order[] arr, int low, int high) {
30     double pivot = arr[high].totalPrice;
31     int i = low - 1;
32
33-     for (int j = low; j < high; j++) {
34-         if (arr[j].totalPrice < pivot) {
35             i++;
36             Order temp = arr[i];
37             arr[i] = arr[j];
38             arr[j] = temp;
39         }
40     }
```

Output

```
Ram 1500.0
John 3000.0
Sam 5000.0

=== Code Execution Successful ===
```

Programiz
Online Java Compiler

Programiz PRO >

Main.java

```
38     arr[j] = temp;
39 }
40
41
42 Order temp = arr[i + 1];
43 arr[i + 1] = arr[high];
44 arr[high] = temp;
45
46 return i + 1;
47 }
48
49- static void quickSort(Order[] arr, int low, int high) {
50     if (low < high) {
51         int pi = partition(arr, low, high);
52
53         quickSort(arr, low, pi - 1);
54         quickSort(arr, pi + 1, high);
55     }
56 }
57
```

Output

```
Ram 1500.0
John 3000.0
Sam 5000.0

=== Code Execution Successful ===
```

Programiz
Online Java Compiler

Programiz PRO >

Main.java

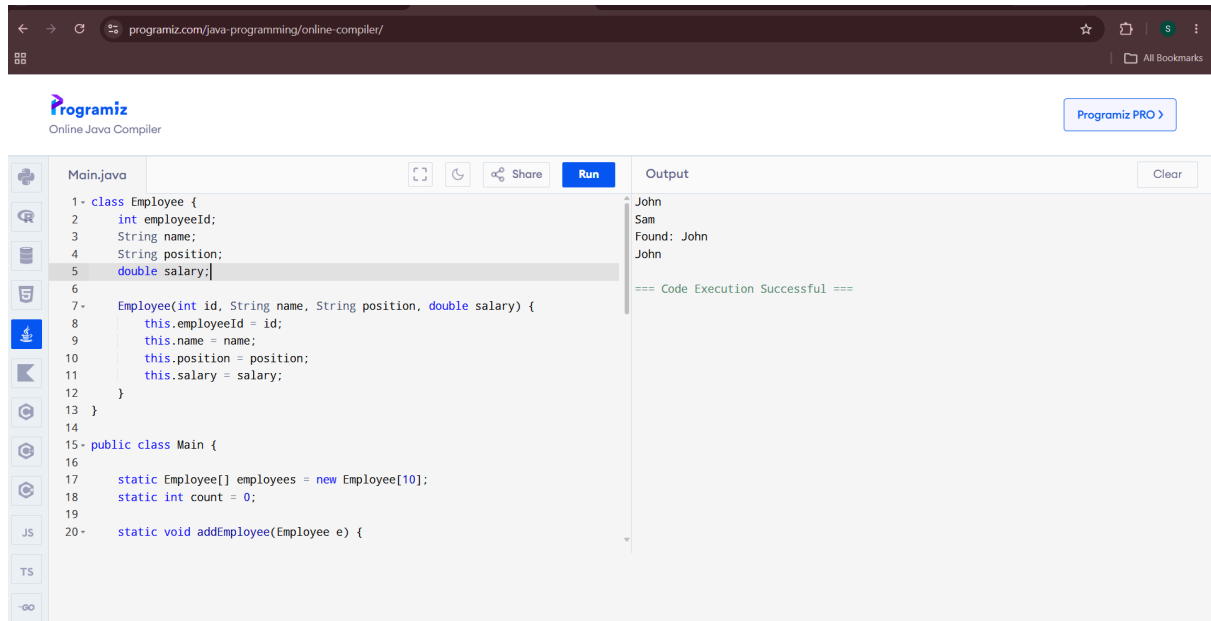
```
52
53     quickSort(arr, low, pi - 1);
54     quickSort(arr, pi + 1, high);
55 }
56 }
57
58- public static void main(String[] args) {
59     Order[] orders = {
60         new Order(1, "John", 3000),
61         new Order(2, "Ram", 1500),
62         new Order(3, "Sam", 5000)
63     };
64
65     quickSort(orders, 0, orders.length - 1);
66
67-     for (Order o : orders) {
68         System.out.println(o.customerName + " " + o.totalPrice);
69     }
70 }
71 }
```

Output

```
Ram 1500.0
John 3000.0
Sam 5000.0

=== Code Execution Successful ===
```

4. Employee Management System

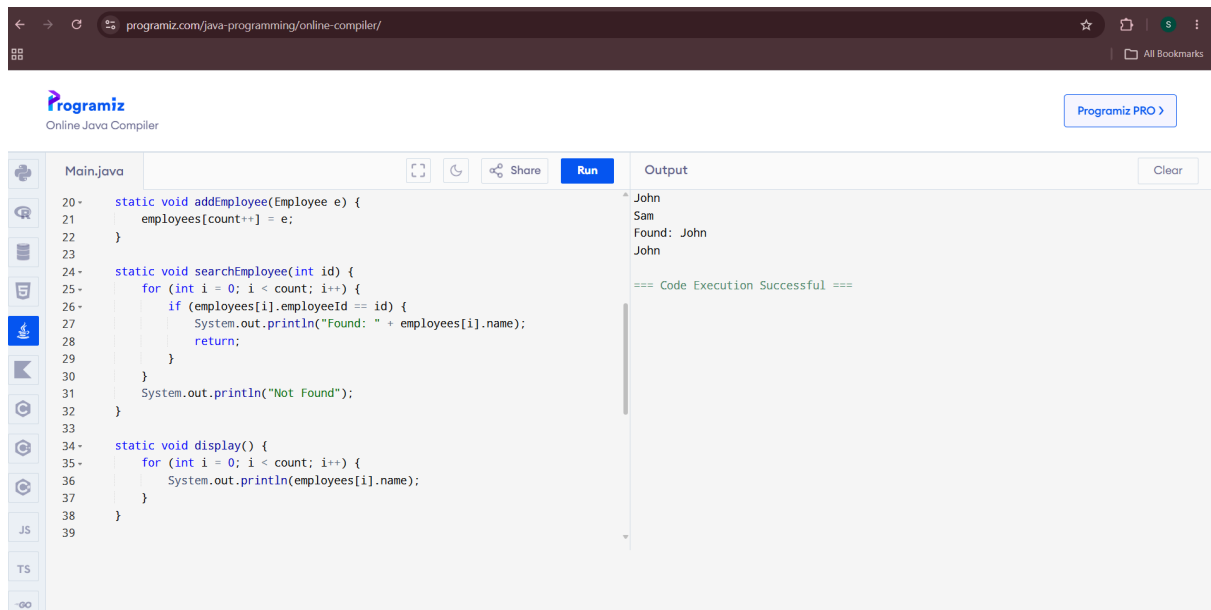


The screenshot shows the Programiz Online Java Compiler interface. The code editor contains the following Java code:

```
1- class Employee {
2-     int employeeId;
3-     String name;
4-     String position;
5-     double salary;
6-
7-     Employee(int id, String name, String position, double salary) {
8-         this.employeeId = id;
9-         this.name = name;
10-        this.position = position;
11-        this.salary = salary;
12-    }
13- }
14-
15- public class Main {
16-
17-     static Employee[] employees = new Employee[10];
18-     static int count = 0;
19-
20-     static void addEmployee(Employee e) {
```

The output window shows the following text:

```
John
Sam
Found: John
John
=== Code Execution Successful ===
```



The screenshot shows the Programiz Online Java Compiler interface with the completed code. The code editor contains the following Java code:

```
20-     static void addEmployee(Employee e) {
21-         employees[count++] = e;
22-     }
23-
24-     static void searchEmployee(int id) {
25-         for (int i = 0; i < count; i++) {
26-             if (employees[i].employeeId == id) {
27-                 System.out.println("Found: " + employees[i].name);
28-                 return;
29-             }
30-         }
31-         System.out.println("Not Found");
32-     }
33-
34-     static void display() {
35-         for (int i = 0; i < count; i++) {
36-             System.out.println(employees[i].name);
37-         }
38-     }
39- }
```

The output window shows the following text:

```
John
Sam
Found: John
John
=== Code Execution Successful ===
```

The screenshot shows the Programiz Online Java Compiler interface. The code in Main.java is as follows:

```
41- for (int i = 0; i < count; i++) {
42-     if (employees[i].employeeId == id) {
43-         for (int j = i; j < count - 1; j++) {
44-             employees[j] = employees[j + 1];
45-         }
46-         count--;
47-     }
48- }
49-
50-
51- public static void main(String[] args) {
52-     addEmployee(new Employee(1, "John", "Manager", 50000));
53-     addEmployee(new Employee(2, "Sam", "Developer", 40000));
54-
55-     display();
56-     searchEmployee(1);
57-     deleteEmployee(2);
58-     display();
59- }
60- }
```

The output on the right shows the results of the program execution:

```
John
Sam
Found: John
John
=== Code Execution Successful ===
```

5.Task Management System (Singly Linked List)

The screenshot shows the Programiz Online Java Compiler interface. The code in Main.java is as follows:

```
1- class Task {
2-     int taskId;
3-     String taskName;
4-     String status;
5-     Task next;
6-
7-     Task(int id, String name, String status) {
8-         taskId = id;
9-         taskName = name;
10-         this.status = status;
11-     }
12- }
13-
14- public class Main {
15-
16-     Task head;
17-
18-     void addTask(int id, String name, String status) {
19-         Task newTask = new Task(id, name, status);
20-     }
21- }
```

The output on the right shows the results of the program execution:

```
Coding
Testing
=== Code Execution Successful ===
```

The screenshot shows the Programiz Online Java Compiler interface. The browser address bar displays `programiz.com/java-programming/online-compiler/`. The compiler has a left sidebar with icons for file operations and a language selector set to Java. The main editor, titled "Main.java", contains the following code:

```
20
21
22     if (head == null)
23         head = new Task();
24     else {
25         Task temp = head;
26         while (temp.next != null)
27             temp = temp.next;
28         temp.next = new Task();
29     }
30
31 void display() {
32     Task temp = head;
33     while (temp != null) {
34         System.out.println(temp.taskName);
35         temp = temp.next;
36     }
37 }
38
39 public static void main(String[] args) {
```

The "Output" pane on the right shows the results of the compilation and execution:

```
Coding
Testing
=== Code Execution Successful ===
```

The screenshot shows the Programiz Online Java Compiler interface with a different Java program. The browser address bar displays `programiz.com/java-programming/online-compiler/`. The compiler has a left sidebar with icons for file operations and a language selector set to Java. The main editor, titled "Main.java", contains the following code:

```
28
29
30
31 void display() {
32     Task temp = head;
33     while (temp != null) {
34         System.out.println(temp.taskName);
35         temp = temp.next;
36     }
37 }
38
39 public static void main(String[] args) {
40     Main list = new Main();
41
42     list.addTask(1, "Coding", "Pending");
43     list.addTask(2, "Testing", "Completed");
44
45     list.display();
46 }
47 }
```

The "Output" pane on the right shows the results of the compilation and execution:

```
Coding
Testing
=== Code Execution Successful ===
```

6. Library Management System

← → ↻ programiz.com/java-programming/online-compiler/ ☆ | S | All Bookmarks

Programiz
Online Java Compiler

Programiz PRO >

Main.java Run Share

```
1- class Book {
2-     int bookId;
3-     String title;
4-     String author;
5-
6-     Book(int id, String title, String author) {
7-         this.bookId = id;
8-         this.title = title;
9-         this.author = author;
10-    }
11- }
12-
13- public class Main {
14-
15-     static int linearSearch(Book[] books, String title) {
16-         for (int i = 0; i < books.length; i++) {
17-             if (books[i].title.equalsIgnoreCase(title))
18-                 return i;
19-         }
20-         return -1;
21-     }
22- }
```

Output Clear

```
Linear Search: 1
Binary Search: 1

=== Code Execution Successful ===
```

Programiz
Online Java Compiler

Programiz PRO >

Main.java Run Share

```
20-     return -1;
21- }
22-
23- static int binarySearch(Book[] books, String title) {
24-     int low = 0, high = books.length - 1;
25-
26-     while (low <= high) {
27-         int mid = (low + high) / 2;
28-
29-         int result = books[mid].title.compareToIgnoreCase(title);
30-
31-         if (result == 0)
32-             return mid;
33-         else if (result < 0)
34-             low = mid + 1;
35-         else
36-             high = mid - 1;
37-     }
38-
39-     return -1;
40- }
```

Output Clear

```
Linear Search: 1
Binary Search: 1

=== Code Execution Successful ===
```

Programiz
Online Java Compiler

Programiz PRO >

Main.java Run Share

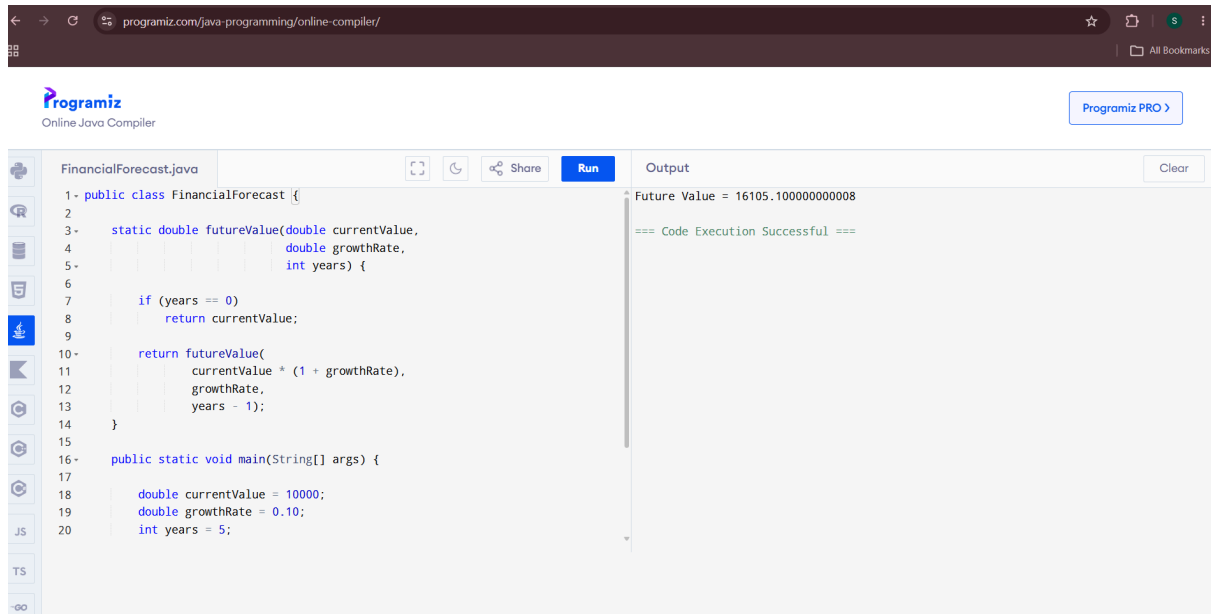
```
36-         high = mid - 1;
37-     }
38-
39-     return -1;
40- }
41-
42- public static void main(String[] args) {
43-     Book[] books = {
44-         new Book(1, "C", "Dennis"),
45-         new Book(2, "Java", "James"),
46-         new Book(3, "Python", "Guido")
47-     };
48-
49-     System.out.println("Linear Search: " +
50-         linearSearch(books, "Java"));
51-
52-     System.out.println("Binary Search: " +
53-         binarySearch(books, "Java"));
54- }
55- }
```

Output Clear

```
Linear Search: 1
Binary Search: 1

=== Code Execution Successful ===
```


7. Financial Forecasting (Recursion)

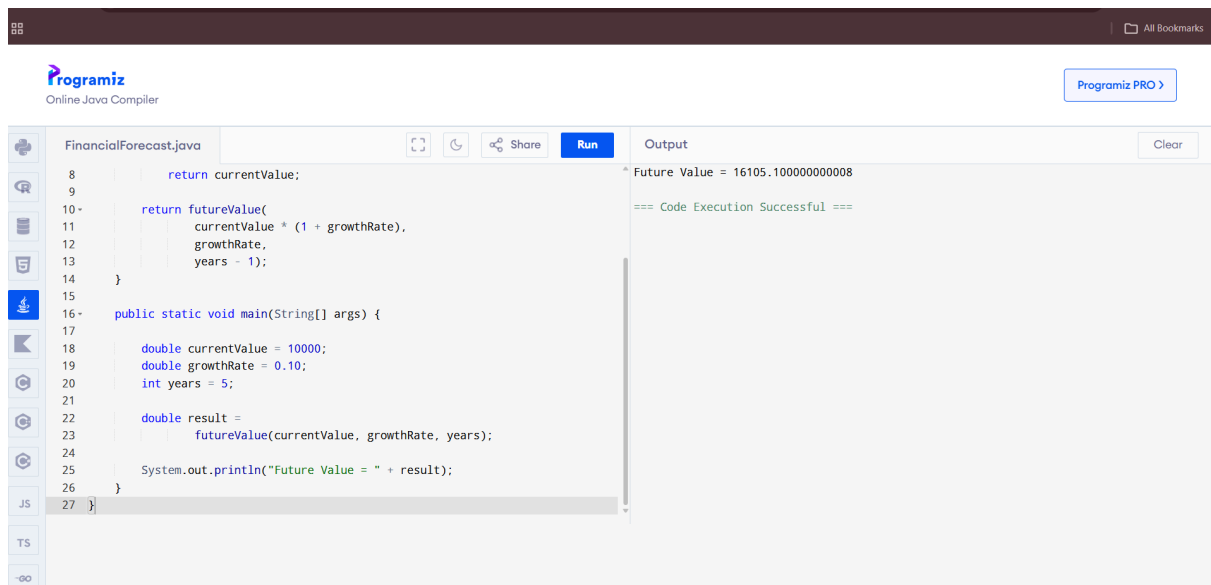


The screenshot shows the Programiz Online Java Compiler interface. The code editor contains a Java program named `FinancialForecast.java`. The program defines a recursive method `futureValue` that calculates the future value of an investment based on the current value, growth rate, and number of years. The `main` method initializes the variables and calls the recursive method.

```
1- public class FinancialForecast {  
2-  
3-     static double futureValue(double currentValue,  
4-                               double growthRate,  
5-                               int years) {  
6-  
7-         if (years == 0)  
8-             return currentValue;  
9-  
10-        return futureValue(  
11-            currentValue * (1 + growthRate),  
12-            growthRate,  
13-            years - 1);  
14-    }  
15-  
16-    public static void main(String[] args) {  
17-  
18-        double currentValue = 10000;  
19-        double growthRate = 0.10;  
20-        int years = 5;  
21-    }  
22-}
```

The output window displays the result of the execution:

```
Future Value = 16105.100000000008  
=== Code Execution Successful ===
```



The screenshot shows the Programiz Online Java Compiler interface. The code editor contains a Java program named `FinancialForecast.java`. The program defines an iterative method `futureValue` that calculates the future value of an investment based on the current value, growth rate, and number of years. The `main` method initializes the variables and calls the iterative method.

```
8-         return currentValue;  
9-  
10-        return futureValue(  
11-            currentValue * (1 + growthRate),  
12-            growthRate,  
13-            years - 1);  
14-    }  
15-  
16-    public static void main(String[] args) {  
17-  
18-        double currentValue = 10000;  
19-        double growthRate = 0.10;  
20-        int years = 5;  
21-  
22-        double result =  
23-            futureValue(currentValue, growthRate, years);  
24-  
25-        System.out.println("Future Value = " + result);  
26-    }  
27-}
```

The output window displays the result of the execution:

```
Future Value = 16105.100000000008  
=== Code Execution Successful ===
```